

CSE 6220 High Performance Computing
Midterm 1 Solutions

1. (a) True. Note the big-O notation defines upper bound.
- (b) False. Brent's lemma work only for $p_2 < p_1$.
- (c) $\frac{3}{4}$
- (d) No.

$$\begin{aligned}
 (a \otimes b) \otimes c &= (2a + b) \otimes c \\
 &= 4a + 2b + c \\
 a \otimes (b \otimes c) &= a \otimes (2b + c) \\
 &= 2a + 2b + c \\
 4a + 2b + c &\neq 2a + 2b + c
 \end{aligned}$$

- (e) Yes. Let $\neg 0$ denote non-zero for c and b in the table below. We can show that operator $\otimes$ is binary associative:

b	c	$(a \otimes b) \otimes c$	$a \otimes (b \otimes c)$
0	0	a	a
0	$\neg 0$	c	c
$\neg 0$	0	b	b
$\neg 0$	$\neg 0$	c	c

2. (a)

$$E_{A_1} = \frac{T(n, 1)}{nT_{A_1}(n, n)} = \frac{\Theta(n^2)}{n\Theta(n)} = \Theta(1)$$

$$E_{A_2} = \frac{T(n, 1)}{n^2T_{A_2}(n, n^2)} = \frac{\Theta(n^2)}{n^2\Theta(\log n)} = \Theta(\frac{1}{\log n})$$

Therefore, A_1 is more efficient.

- (b)

$$S_{A_1} = \frac{T(n, 1)}{T_{A_1}(n, n)} = \frac{\Theta(n^2)}{\Theta(n)} = \Theta(n)$$

$$S_{A_2} = \frac{T(n, 1)}{T_{A_2}(n, n^2)} = \frac{\Theta(n^2)}{\Theta(\log n)} = \Theta(\frac{n^2}{\log n})$$

Therefore, A_2 gives more speed.

- (c) A_1 is more practical since there would probably be fewer processors.
- (d) A_1 is more practical since it consumes less memory.

3.

$$E(p) = \frac{T(n, 1)}{pT(n, p)}$$

Optimal Efficiency occurs when $E(p) = \Theta(1)$

$$\frac{n}{n + p^{3/2} \log(p)} = \Theta(1)$$

$$p^{3/2} \log(p) = O(n)$$

$$\frac{2}{3} p^{3/2} \log(p^{3/2}) = O(n)$$

$$p = O\left(\frac{n^{2/3}}{\log^{2/3}(n)}\right)$$

4. Let A be the input sequence. Create an array B according to the following rule.

$$B_i = \begin{cases} +1 & \text{if } A_i = '(' \\ -1 & \text{if } A_i = ')' \end{cases}$$

Run parallel prefix sum on B and store the results in an array C . A well-formed sequence A must have every element of C non-negative and the final element of C must be 0. Locally check if these conditions hold and store a corresponding boolean value b_i for integer $j \in [0, n - 1]$.

$$b_i = \begin{cases} \text{true} & \text{if } i < n - 1 \text{ and } C_i \geq 0 \\ \text{true} & \text{if } i = n - 1 \text{ and } C_i = 0 \\ \text{false} & \text{otherwise} \end{cases}$$

The boolean results can be combined using a parallel reduction with boolean AND operator.

B can be computed locally by each processor in $\Theta(\frac{n}{p})$ time, and parallel prefix will dominate the runtime.

Thus, the overall runtime is $\Theta((\tau + \mu) \log(p))$ for communication and $\Theta(\frac{n}{p} + \log(p))$ for computation.

5. Perform parallel prefix on A using the operator defined in problem 1e. Runtime is the same as parallel prefix.

Alternative solution:

Let

$$B[i] = \begin{cases} 0 & \text{if } A[i] = 0 \\ i & \text{otherwise} \end{cases}$$

Define a tuple $[B[i], A[i]]$ for every $i \in [0, n - 1]$, and perform the parallel prefix on elements $[B[i], A[i]]$ using the operation $\otimes$ defines as:

$$\begin{bmatrix} a_1 & a_2 \end{bmatrix} \otimes \begin{bmatrix} b_1 & b_2 \end{bmatrix} = \begin{cases} \begin{bmatrix} a_1 & a_2 \end{bmatrix} & \text{if } a_1 > b_1 \\ \begin{bmatrix} b_1 & b_2 \end{bmatrix} & \text{otherwise} \end{cases}$$

Essentially, we are performing the parallel prefix max on $B[i]$ but we also send the corresponding $A[i]$ element along the way although the operator $\otimes$ does not depend on $A[i]$.

Let the resulting tuple be $C[i] = \begin{bmatrix} c_{i1} & c_{i2} \end{bmatrix}$. For each i where $A[i] = 0$, replace $A[i]$ with c_{i2} .